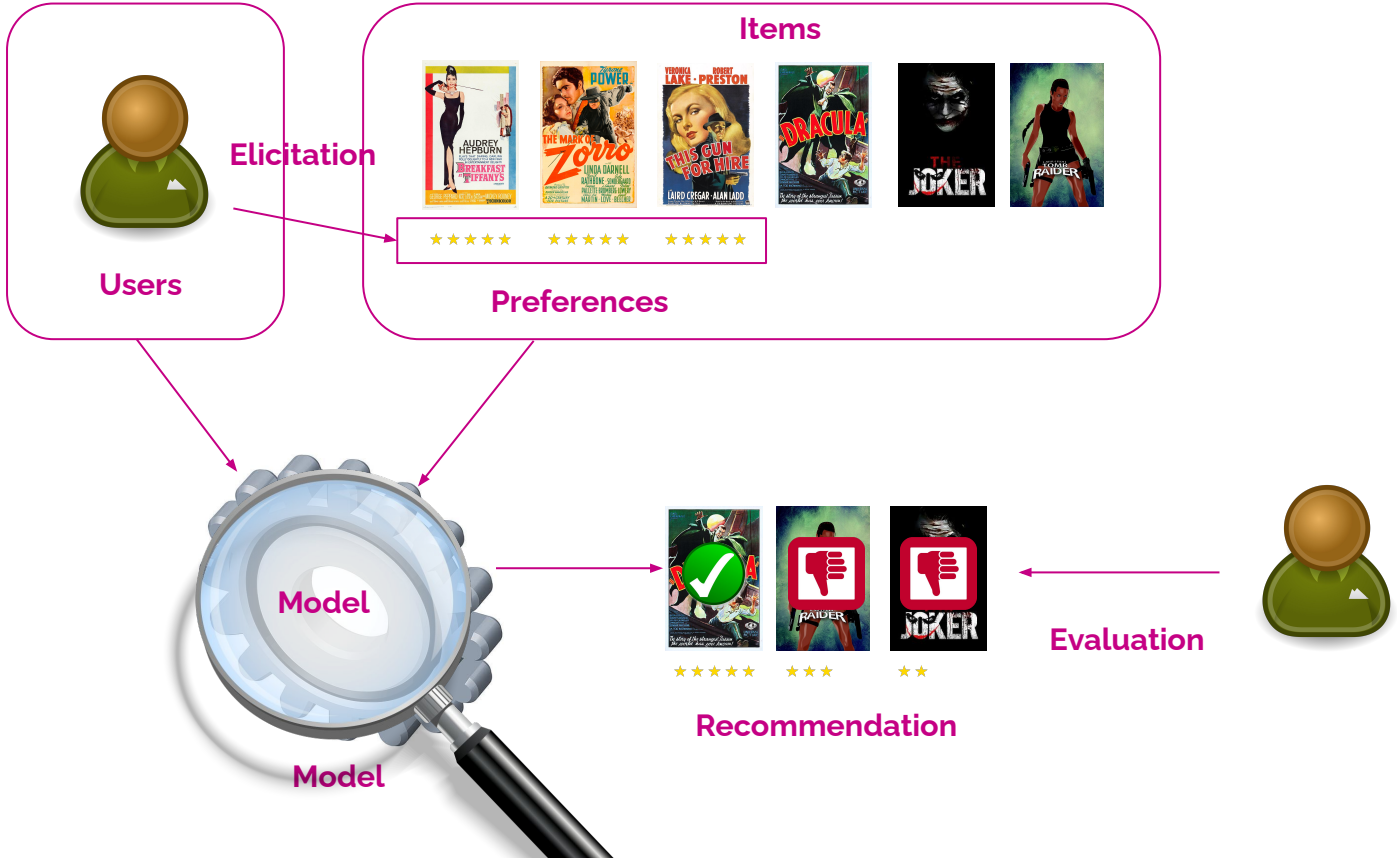
UNIVERSIDADE DA CORUÑA

Recommendation Models: Collaborative Filtering (ii)

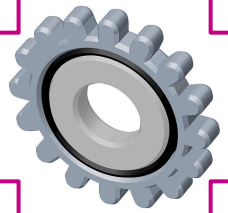
Javier Parapar López

Recommender System



Non personalized

- Most popular
- Most recent
- Best rated
- Best sold
- Most margin

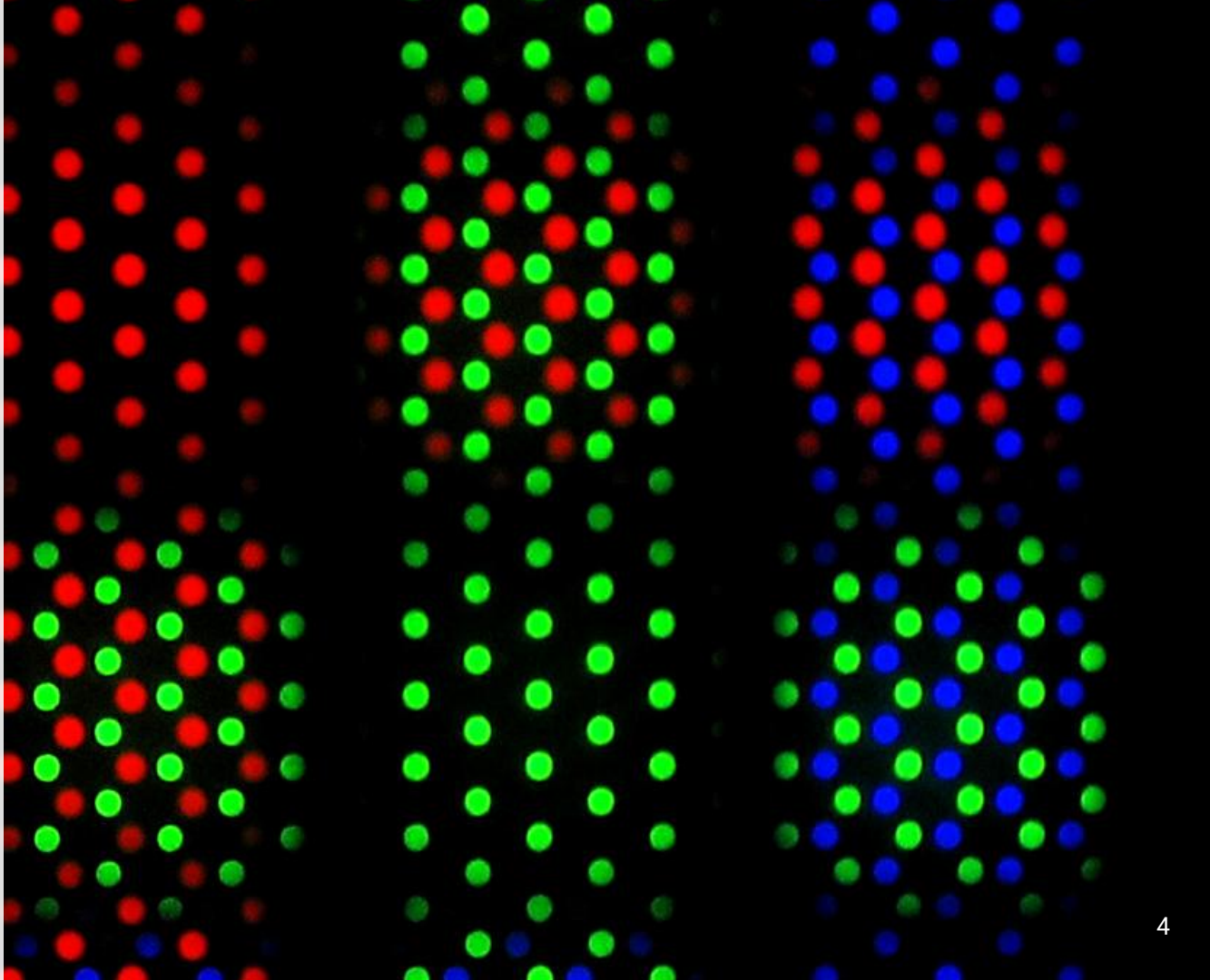


Personalized

- Content Based
- Collaborative Filtering
  - User Based
  - Item Based
  - Matrix Factorization
  - Others
- CARS
- TARS
- Hybrid

Matrix Factorization

- Decompose preference matrix in order to obtain dense representations where **latent factors** serve both for expressing preferences over and represent item features.



Basic idea

- The idea is to have both user and items represented in a common feature space with so many **dimensions as** the number of **latent factors.**
- The user vector contains the **interest** of the **user** in each of the factors and the item vector how good that factor **describes** the **item**.

$$\vec{x}_u = \begin{array}{c} x_{uk} \\ \begin{array}{|c|c|c|c|c|c|c|} \hline & & & & & & \\ \hline \end{array} \end{array}$$

$$\vec{y}_i = \begin{array}{c} y_{ik} \\ \begin{array}{|c|c|c|c|c|c|c|} \hline & & & & & & \\ \hline \end{array} \end{array}$$

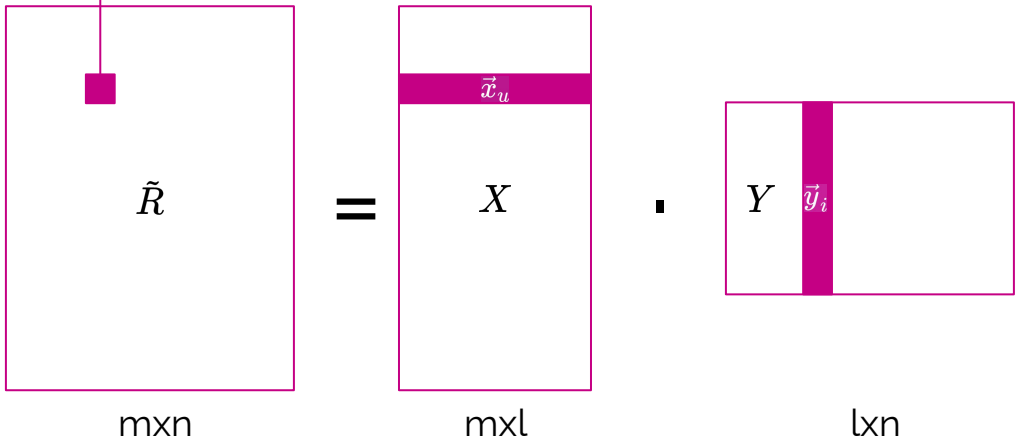
$$\tilde{r}_{u,i} = \sum_k x_{uk} \cdot y_{ik} = \vec{x_u} \cdot \vec{y}_i$$

How can we compute the latent space?

Matrix decomposition

- l = number of latent factors
- m=|U|
- n=|I|
- X: user latent matrix
- Y: item latent matrix

$$\tilde{r}_{u,i} = \sum_k x_{uk} \cdot y_{ki} = \vec{x}_u \cdot \vec{y}_i$$



Optimization problem

$$\tilde{r}_{u,i} = \sum_k x_{uk} \cdot y_{ik} = \vec{x_u} \cdot \vec{y_i}$$

$$\tilde{R} = X \cdot Y \longrightarrow X^*, Y^* = \min_{X,Y} \left(\left\| R - \bar{R} \right\|_2 \right)$$

How many latent factors?

l large |R| small	l small |R| big
Increased risk of overfitting → bad recommendations	Low personalization biased to poplity (l=1 is popularity)

- X^* and Y^* are the ideal matrices which minimize the error between the estimated and the actual preference matrix.
- We may obtain good approximations (X,Y) of the ideal matrices by solving a minimization problem, e.g. the 2-norm minimizes the RMSE.

Regularization of the problem

$$X^*, Y^* = \min_{X,Y}\left(\left\|R - \bar{R}\right\|_2 + \boxed{\lambda_1\|X\| + \lambda_2\|Y\|}\right)$$

Regularization terms

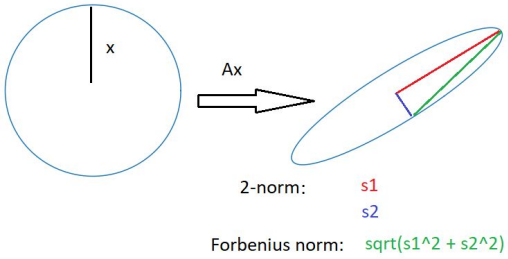
- By adding the regularization terms we reduce the overfitting problem by favouring sparsity in the matrices

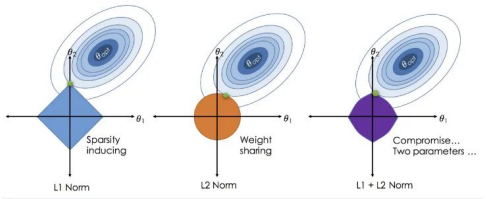
Regularization parameter values

$\lambda_1 \lambda_2$ small	$\lambda_1 \lambda_2$ big
overfit	simpler model X and Y will be too sparse (mostly zeros)

Missing preferences

Missing as random	Missing as negative
We assume that the probability of an user liking or disliking a non rated item is equal → **2-norm (spectral norm in matrices)** measures the maximum stretching the matrix applies to any vector. *This is **not true** we know that users tend to rate more items that they like. e.g. funkMF	We assume that items without preferences are more prone to be not liked by the user → **Frobenius norm** The Frobenius norm is like the 2-norm but computed also over missing ratings. Treat the matrix like a long vector sum the squares of all entries e.g Bayesian Personalized Ranking



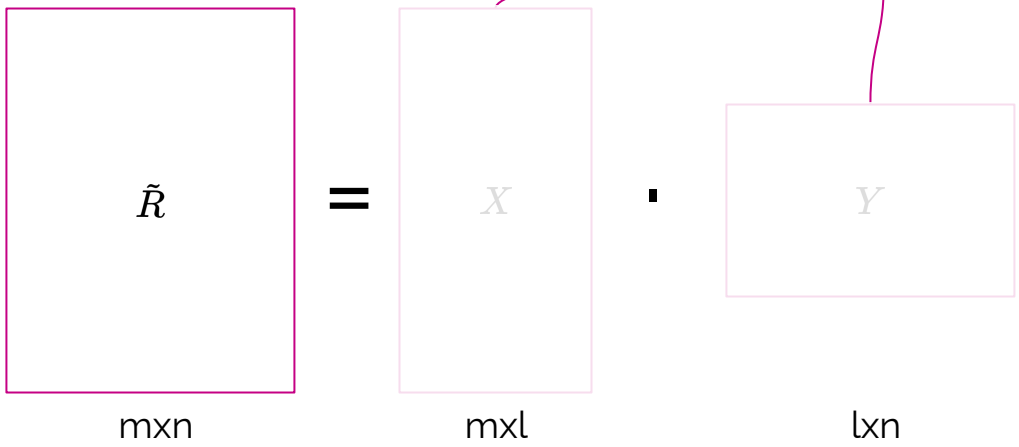


ALS: Alternating Least Squares

$$\tilde{R} = X \cdot Y$$

$$X^*, Y^* = \min_{X,Y}(\|R - XY\|_2 + \lambda_1\|X\| + \lambda_2\|Y\|)$$ → No analytical solution

Initialized randomly



Fix X learn Y

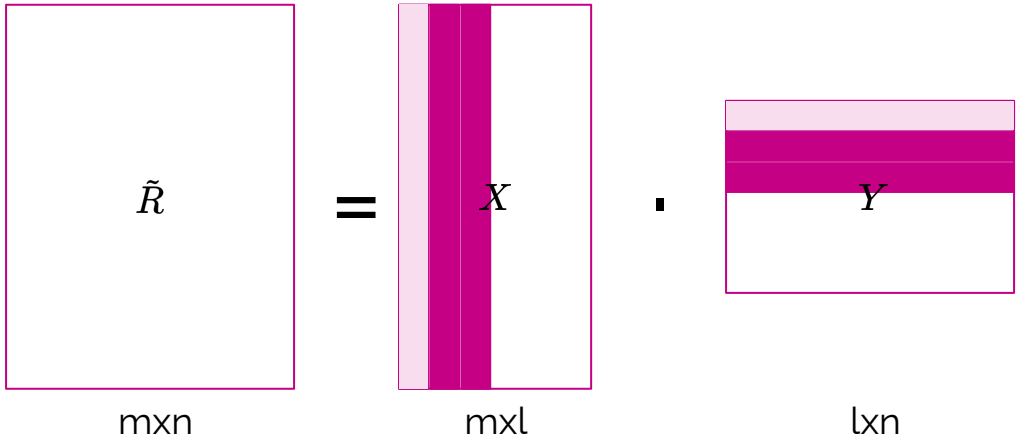
Fix Y learn X

and so and so:

- Until the error is minimized
- No significant improvements

Simon Funk Matrix Factorization

Computes each latent factor sequentially with ALS



$$\tilde{r}_{u,i} = \sum_k x_{uk} \cdot y_{ki} = \vec{x_u} \cdot \vec{y_i}$$

SVD++

Incorporates global effects for coping better with the noise of explicit ratings

$$\tilde{r}_{u,i} = \boxed{\mu} + \boxed{b_u} + \boxed{b_i} + \sum_k x_{uk} \cdot y_{ki}$$

Overall mean rating of R

User bias: average user rating

Item bias: average item rating

SVD++

$$\tilde{r}_{u,i} = \mu + b_u + b_i + \sum_k x_{uk} \cdot y_{ki}$$

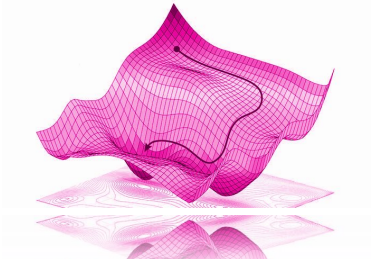
$$\downarrow$$

$$\mu^*, b_u^*, b_i^*, X^*, Y^* = \min_{X,Y}(\|R - XY\|_2 + \lambda_1\|X\| + \lambda_2\|Y\|)$$

It is not possible to use ALS due the global factors:

- The prediction for one rating depends on all items the user interacted with, and the parameters y_k affect many users simultaneously.
- This breaks the ALS property that parameters can be solved independently in closed form.

→ SGD (Stochastic Gradient Descent)



SGD for RS

$$\tilde{r}_{u,i} = \mu + b_u + b_i + \sum_k x_{uk} \cdot y_{ki}$$

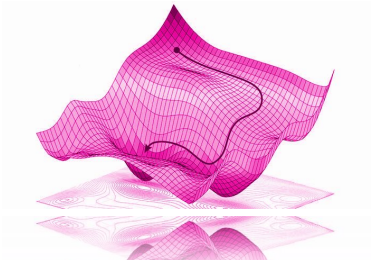
$$\downarrow$$

$$\mu^*, b_u^*, b_i^*, X^*, Y^* = \min_{X,Y}(\|R - XY\|_2 + \lambda_1\|X\| + \lambda_2\|Y\|)$$

It is not possible to use ALS due the global factors:

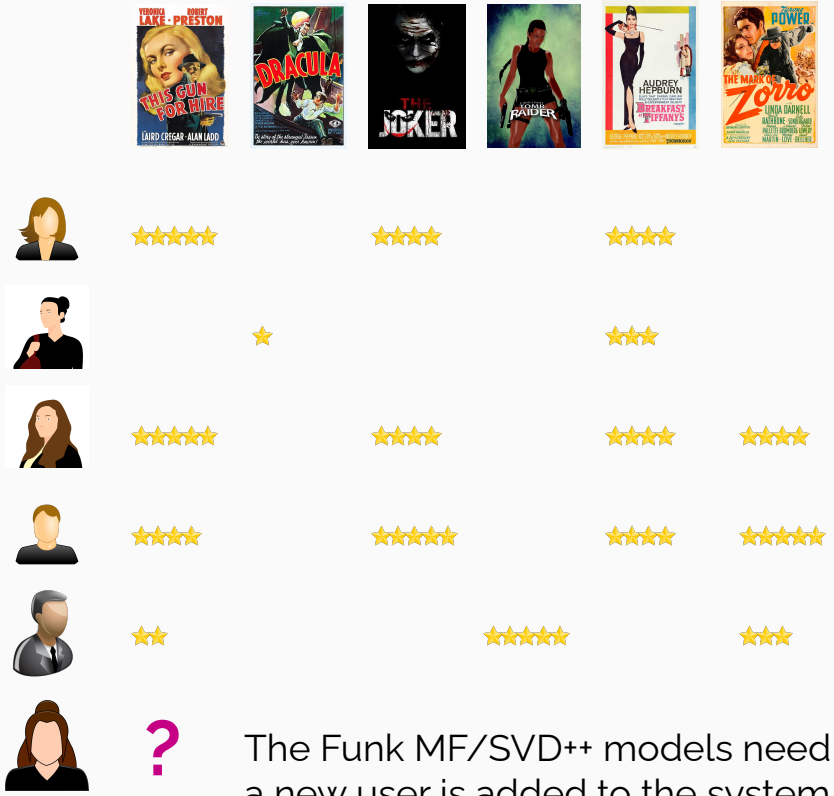
- The prediction for one rating depends on all items the user interacted with, and the parameters y_k affect many users simultaneously.
- This breaks the ALS property that parameters can be solved independently in closed form.

→ SGD (Stochastic Gradient Descent)



SGD for RecSys

- SGD minimizes loss by updating parameters iteratively
- It uses the gradient computed from only one training example (**one rating from u to i**)
- The update rule moves parameters to reduce the prediction error
- SGD is effective for large sparse datasets in Recommender Systems
- It allows incremental updates

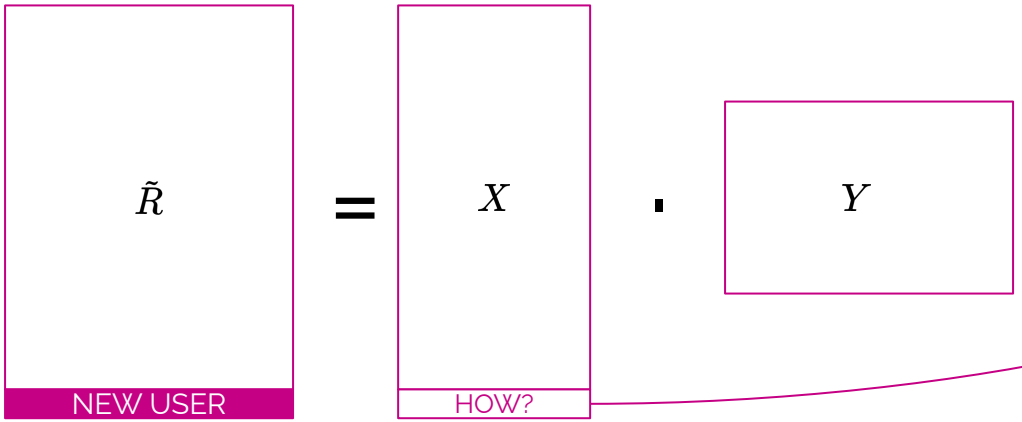


?

The Funk MF/SVD++ models need to be retrained every time a new user is added to the system → Asymmetric SVD

Asymmetric SVD

$$\tilde{r}_{u,i} = \sum_k x_{uk} \cdot y_{ki} = \vec{x_u} \cdot \vec{y_i}$$



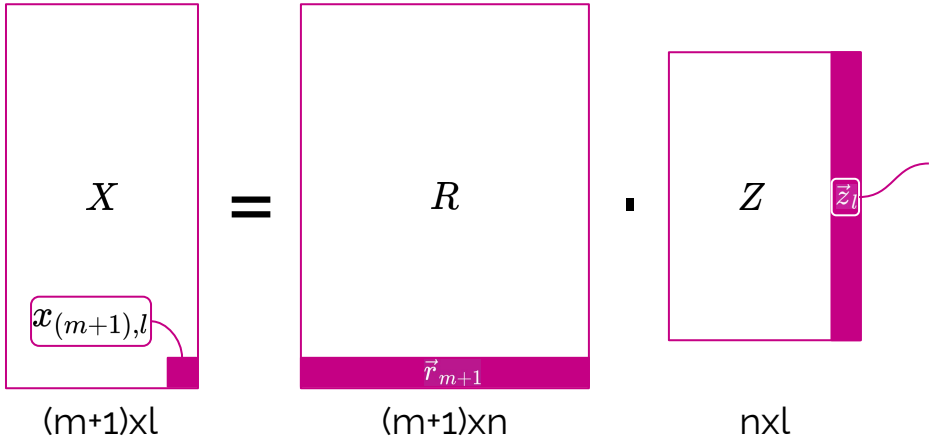
(m+1)xn (m+1)xl lxn

How much latent feature *k* is present in item *j*

$$x_{uk} = \sum_j r_{u,j} \cdot \boxed{z_{jk}} = \vec{r_u} \cdot \vec{z_k}$$

How compute Z?

$$X = R \cdot Z$$

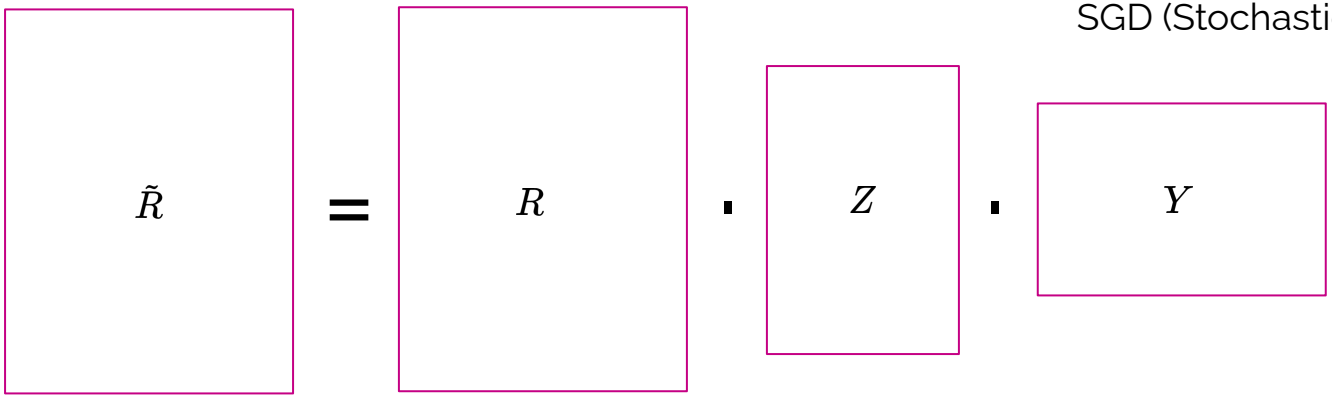


Latent feature vector for dimension *l*

How compute Z?

$$\bar{R} = R \cdot Z \cdot Y \longrightarrow \tilde{r}_{u,i} = \sum_k \sum_j r_{u,j} \cdot z_{jk} \cdot y_{ki} \longrightarrow \tilde{r}_{u,i} = \mu + b_u + b_i + \sum_k \sum_j r_{u,j} \cdot z_{jk} \cdot y_{ki}$$

SGD (Stochastic Gradient Descent)



mxn mxn nxl lxn

Factorization meets the neighborhood: a multifaceted collaborative filtering model

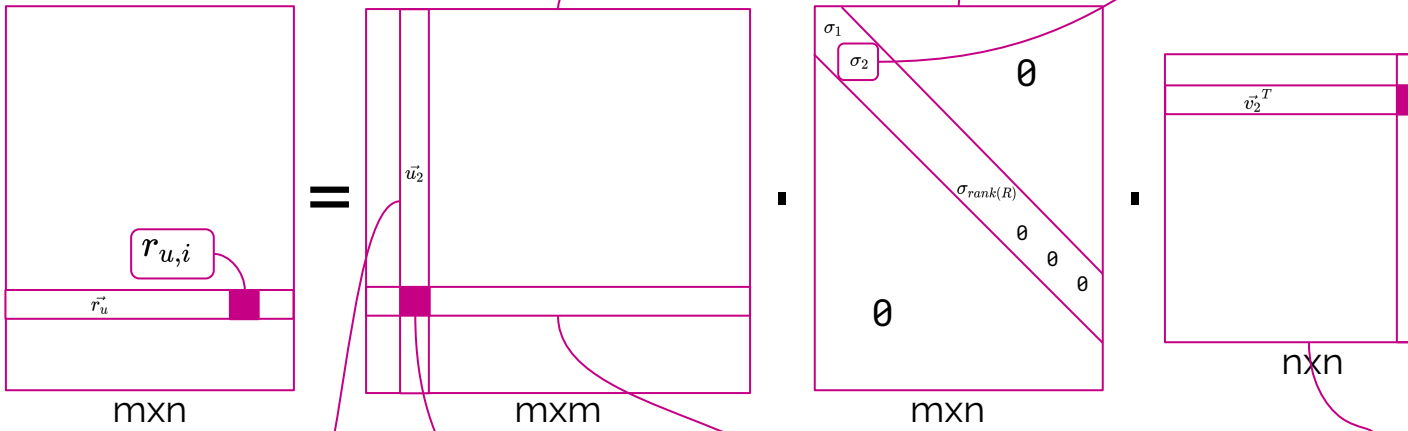
SVD

$$R = U \cdot \Sigma \cdot V^T$$

Columns: **Left singular vectors (RR^T)u=λu**
λ_i is the eigenvalue (the square of σ_i, the singular value)
Ortogonal $U^TU = I$ (identity matrix)
Eigenvectors of RR^T: describe the variance in the **R columns**
Left-to-right: the leftmost vector is the one best describing R

Non-negative diagonal matrix of rank equal to the rank of R
Diagonal contains the **singular values** or R in decreasing magnitude order $\det(RR^T-\lambda I)=0$, $\sigma^2=\lambda$

The strength of feature 2 in describing items (singular value)



Features in rows

How much feature 2 describes item *i*

Items in columns

Features in columns

How much user *u* is interested in feature 2

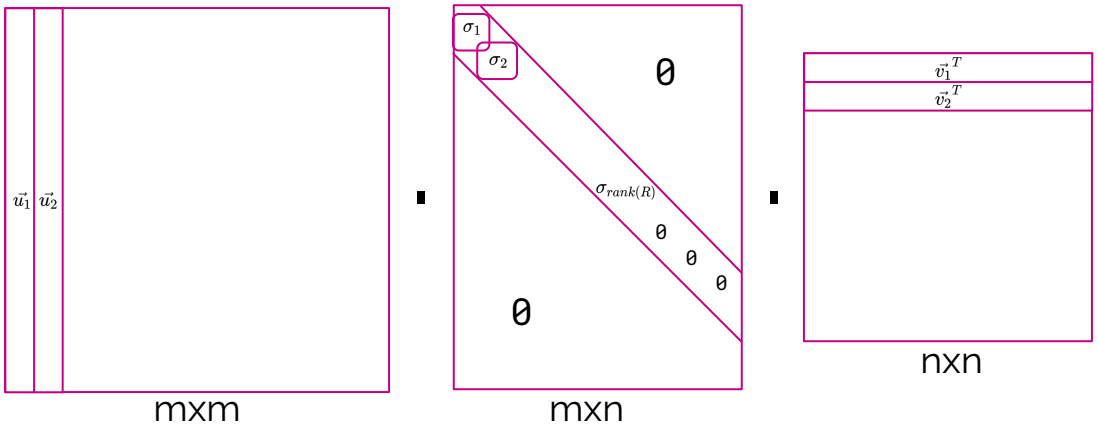
Users in rows

Rows: **Right singular vectors (R^TR)v=λv**
Ortogonal
Eigenvectors of R^TR: describe the variance in the **R rows**
Top-to-bottom: the top vector is the one best describing R

SVD Computation

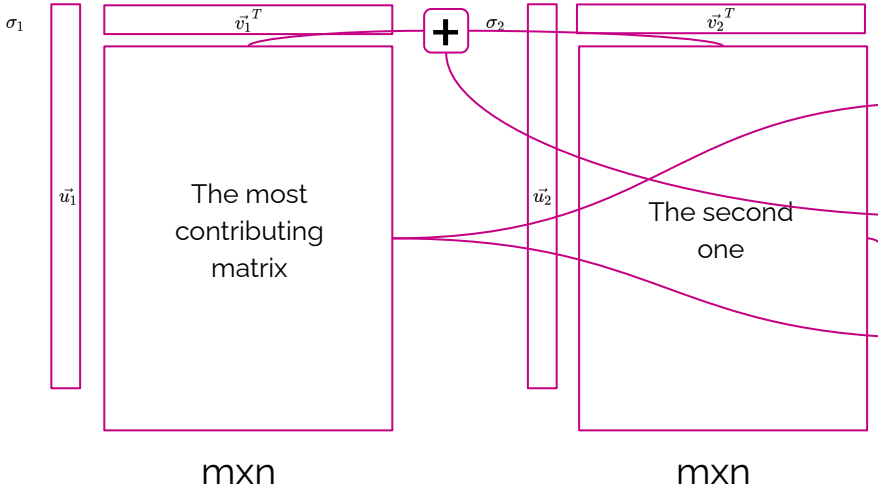
$$R = U \cdot \Sigma \cdot V^T = \sigma_1 \vec{u_1} \vec{v_1}^T + \sigma_2 \vec{u_2} \vec{v_2}^T + \ldots \longrightarrow R = U \cdot \Sigma \cdot V^T = \sum_{k=1}^{rank(R)} \sigma_k \, \vec{u_k} \, \vec{v_k}^T + \boxed{\sum_{k=rank(R)+1}^{\min(m,n)} 0 \, \vec{u_k} \, \vec{v_k}^T}$$

$$\boxed{0}$$



Truncated SVD

$$R = U \cdot \Sigma \cdot V^T = \sigma_1 \vec{u_1} \vec{v_1}^T + \sigma_2 \vec{u_2} \vec{v_2}^T + \ldots \longrightarrow R = U \cdot \Sigma \cdot V^T = \sum_{k=1}^{rank(R)} \sigma_k \, \vec{u_k} \, \vec{v_k}^T + \sum_{k=rank(R)+1}^{\min(m,n)} 0 \, \vec{u_k} \, \vec{v_k}^T$$



...

Best rank-1 approximation: $\min_{\tilde{R}:rank(\tilde{R})=1} \left\| R - \tilde{R} \right\|_F : \sigma_1 \, \vec{u_1} \, \vec{v_1}^T$

Best rank-2 approximation, and so on

Each matrix has rank 1, that is, only one column and row from each are linearly independent

Truncated at rank k is the rank-k approximation

$$\min_{\tilde{R}:rank(\tilde{R})=k} \left\| R - \tilde{R} \right\|_F : \sum_{i=1}^{k} \sigma_i \, \vec{u_i} \, \vec{v_i}^T$$

Pure SVD

$$R = U \cdot \Sigma \cdot V^T \approx \boxed{\tilde{U}} \cdot \tilde{\Sigma} \cdot \boxed{\tilde{V^T}} = \sum_{i=1}^{k} \sigma_i \, \vec{u_i} \, \vec{v_i}^T$$

The approximated matrices are not any more orthogonal, only column orthogonal:

$$\tilde{U^T}\tilde{U} = \tilde{V^T}\tilde{V} = I_{rank(R)\text{x}rank(R)} \neq I$$

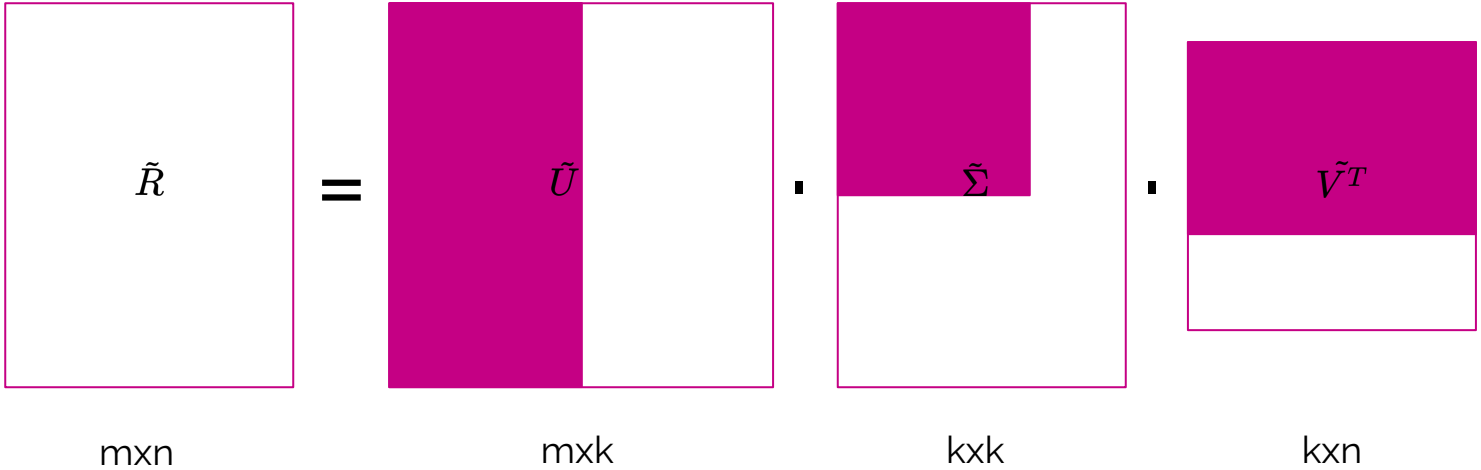
$$R \approx \tilde{R} = \tilde{U} \cdot \tilde{\Sigma} \cdot \tilde{V^T}$$

In Pure SVD the result of the truncated SVD is used as estimation of the user preferences:

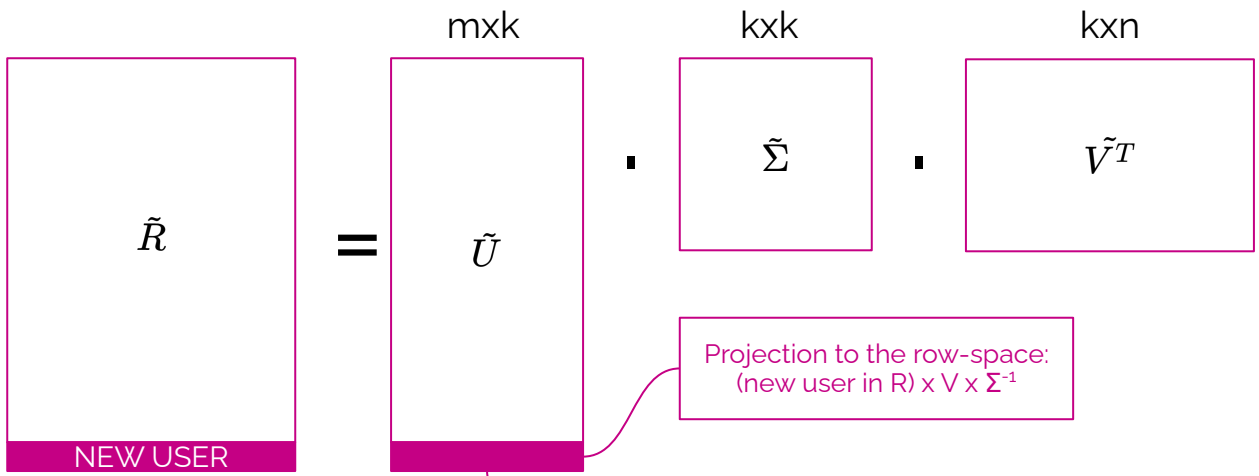
- Helps keeping only the main latent features
- Reduces overfitting.

What happens when we do not truncate and try to predict with the un-truncated SVD a missing preference?

Pure SVD



Incremental SVD



Projection to the row-space:
(new user in R) x V x Σ^{-1}

Folding-in, then compute the new user estimated preference vector in the basis of user latent space

Incremental Singular Value Decomposition Algorithms for Highly Scalable Recommender Systems

Using Linear Algebra for Intelligent Information Retrieval Page 6

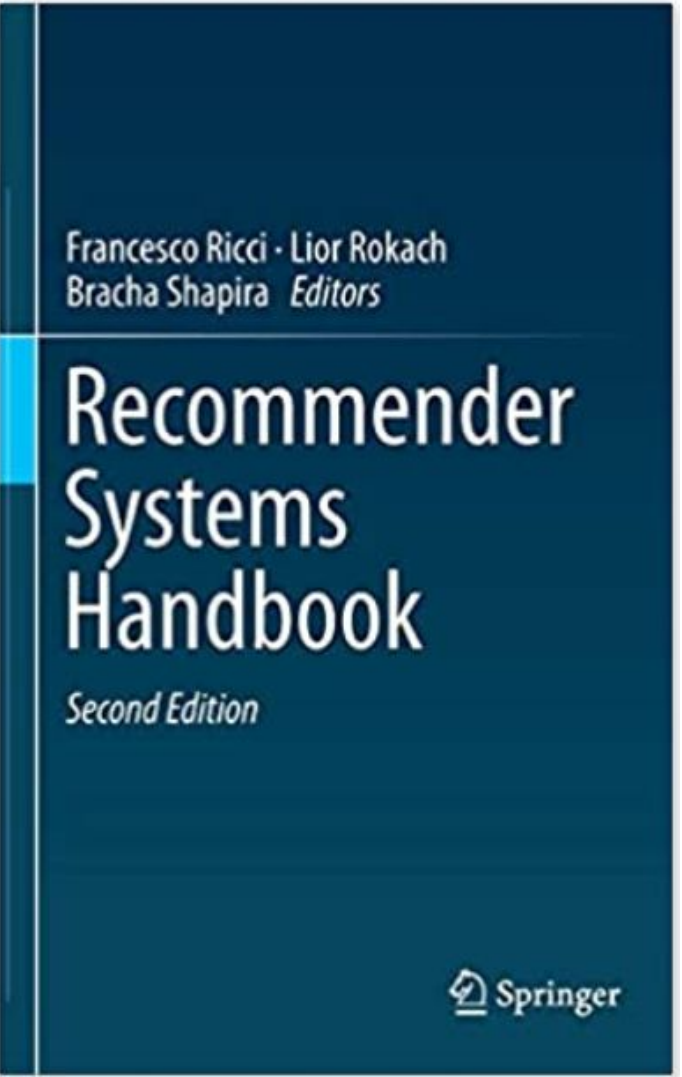
Recap

Effects of
missing at random
vs
missing as negative

funkMF

Asymmetric SVD

Pure SVD



Your turn:

- Recommender Systems Handbook
  - *Chapter 5:*
    - *5.3 Matrix factorization models*
- *Recommender Systems Textbook*
  - *Chapter 3:*
    - *3.6 Latent Factor Models*

UNIVERSIDADE DA CORUÑA

Recommendation Models: Collaborative Filtering (ii)

Grado en Ciencia e Ingeniería de Datos

based on materials from D. Jannach, G. Friedrich, P. Cremonesi, R Sami, and others.